

ĐẠI HỌC QUỐC GIA TP HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

Proof of Concept

Đề tài: TourXport - Smart Tourism System

Môn học: Tư duy tính toán

Sinh viên thực hiện:

Phạm Anh Tuấn - 24120238

Nguyễn Anh Đức - 24120289

Nguyễn Đức Anh Khôi - 24120076

Võ Minh Khang - 24120336

Nguyễn Quốc Nam - 24120098

Phan Văn Việt - 24120244

Nguyễn Anh Thái - 24120224

Giáo viên hướng dẫn:

ThS. Nguyễn Trọng Việt

Ngày 16 tháng 5 năm 2026



Danh sách thành viên

MSSV	Họ và tên	Vai trò	Nhiệm vụ
24120238	Phạm Anh Tuấn	Frontend developer Project Manager	Thiết kế giao diện và phát triển app Flutter. Quản lý tiến độ dự án. Viết báo cáo Chương 1.
24120289	Nguyễn Anh Đức	Frontend developer	Thiết kế giao diện và phát triển app Flutter. Viết báo cáo phần 2 Chương 3.
24120098	Nguyễn Quốc Nam	Backend developer DevOps Engineer	Thiết kế và phát triển Backend. Cấu hình môi trường, Docker và triển khai hệ thống lên Server. Tổng hợp báo cáo
24120244	Phan Văn Việt	Backend developer	Thiết kế và phát triển Backend. Viết báo cáo Chương 2 và phần 1 Chương 3.
24120076	Nguyễn Đức Anh Khôi	AI developer	Thiết kế và phát triển mô hình AI. Viết báo cáo Chương 4 và phần 3 Chương 5.
24120336	Võ Minh Khang	AI developer	Thiết kế và phát triển mô hình AI. Viết báo cáo Chương 4 và phần 3 Chương 5.
24120224	Nguyễn Anh Thái	Data Engineer	Thu thập, xử lý và phân tích dữ liệu. Viết báo cáo phần 4 Chương 3.

Bảng 1: Danh sách các thành viên và nhiệm vụ

Mục lục

CHƯƠNG 1: TỔNG QUAN VÀ PHÂN TÍCH YÊU CẦU	1
1 Giới thiệu dự án	1
2 Phân tích bài toán	1
3 Mục tiêu và phạm vi POC	2
3.1 Mục tiêu POC	2
3.2 Phạm vi POC	2
CHƯƠNG 2: KIẾN TRÚC HỆ THỐNG TỔNG THỂ	4
1 Sơ đồ kiến trúc tổng quát	4
2 Luồng giao tiếp giữa các thành phần	5
3 Thiết kế Cơ sở dữ liệu tập trung trên MongoDB	6
3.1 Cấu trúc người dùng (User Collection).	6
3.2 Cấu trúc hành trình (Tour Collection).	6
3.3 Cấu trúc địa điểm (Place Collection).	6
CHƯƠNG 3: CHI TIẾT CÁC THÀNH PHẦN CỐT LÕI	8
1 Backend (Node.js/Express & Logic Engine).	8
1.1 Vai trò và Nhiệm vụ	8
1.2 Tech Stack & Thư viện	8
1.3 Quy trình xử lý	9
1.4 Kết quả đạt được	9
2 Frontend (Flutter Multi-platform).	12
2.1 Vai trò và Nhiệm vụ	12
2.2 Tech Stack & Thư viện	12
2.3 Quy trình xử lý	13
2.4 Phân chia công việc.	14
2.4.1 Sprint 1	14

2.4.2 Sprint 2	15
2.4.3 Sprint 3	17
3 AI Worker (LLM & RAG Engine)	18
3.1 Vai trò và Nhiệm vụ	18
3.2 Tech Stack & Thư viện	18
3.3 Quy trình xử lý	19
4 Thu thập Dataset (Data Crawler/Scraper)	20
4.1 Vai trò và Nhiệm vụ	20
4.1.1 Vai trò	20
4.1.2 Nhiệm vụ	20
4.2 Tech Stack & Thư viện	21
4.3 Quy trình xử lý	21
CHƯƠNG 4: THỰC THI VÀ ĐÁNH GIÁ THỰC NGHIỆM	23
1 Xây dựng môi trường và triển khai	23
2 Kết quả thực thi các Use Case	24
2.1 Use Case Đăng nhập và Xác thực	24
2.2 Use Case Khảo sát nhu cầu người dùng (Survey)	25
2.3 Đánh giá độ chính xác và hiệu năng hệ thống	26
2.3.1 Hiệu năng hệ thống và độ trễ	26
2.3.2 Độ chính xác và tính toàn vẹn dữ liệu	27
2.3.3 Tính mô-đun và khả năng mở rộng	27
CHƯƠNG 5: TỔNG KẾT VÀ HƯỚNG PHÁT TRIỂN	28
1 Tổng kết	28
2 Hướng phát triển trong tương lai	29
2.1 Mở rộng hệ sinh thái dịch vụ (Dịch vụ tích hợp)	29
2.2 Hỗ trợ an toàn và tình huống khẩn cấp	29
2.3 Hoàn thiện hậu hành trình	29
3 Tài liệu tham khảo	30

Danh sách hình vẽ

2.1	Kiến trúc thư mục của dự án TourXport	5
2.2	Sơ đồ luồng giao tiếp giữa các thành phần của hệ thống TourXport	6
2.3	Sơ đồ thiết kế cơ sở dữ liệu trên MongoDB	7
3.1	Giao diện quản lý MongoDB Atlas với các Collection đã thiết lập	10
3.2	Bảng các Endpoints và trạng thái hoàn thiện của API	11
3.3	Kết quả kiểm thử API đăng nhập thành công với JWT được trả về	11
3.4	Bảng theo dõi công việc front-end sprint 1	14
3.5	Thiết kế landing page, sign in, sign up	15
3.6	Bảng theo dõi công việc front-end sprint 2	15
3.7	Thiết kế dashboard và places	16
3.8	Bảng theo dõi công việc front-end sprint 3	17
3.9	Màn hình máy ảo profile và trang thông tin cá nhân, email	17
4.1	Sequence Diagram mô tả use case Đăng nhập và xác thực	24
4.2	Màn hình câu hỏi Survey - Mô phỏng trên máy ảo Android	25
4.3	Kết quả từ Survey - Mô phỏng trên máy ảo Android	26

CHƯƠNG 1: TỔNG QUAN VÀ PHÂN TÍCH YÊU CẦU

1 Giới thiệu dự án

TourXport là ứng dụng du lịch thông minh, giúp người dùng khám phá và tìm kiếm những địa điểm tham quan ở Việt Nam một cách cá nhân hóa. Dựa trên sở thích và điều kiện của từng người, ứng dụng gợi ý các điểm đến phù hợp, tích hợp bản đồ định vị và xây dựng lộ trình di chuyển tối ưu. Ứng dụng được xây dựng theo định hướng Flutter [1], bám theo các chuẩn giao diện hiện đại của Material Design 3 [2] để đảm bảo tính nhất quán trên nhiều nền tảng.

Điểm khác biệt của TourXport là kết hợp dữ liệu bản đồ, thời tiết và hướng dẫn di chuyển trong cùng một hệ thống, mang lại trải nghiệm du lịch trọn vẹn và thuận tiện hơn. Phần giao diện được triển khai bằng Dart [3] với phong cách thiết kế trực quan, trong đó các hiệu ứng trong suốt lấy cảm hứng từ glassmorphism [4] được dùng cho một số màn hình chính.

2 Phân tích bài toán

Hiện nay, người dùng khi lên kế hoạch du lịch thường phải sử dụng nhiều ứng dụng khác nhau để tìm kiếm địa điểm, tra cứu bản đồ và tìm đường di chuyển. Điều này gây mất thời gian, thiếu tính đồng bộ và làm giảm trải nghiệm người dùng.

Ngoài ra, việc lựa chọn địa điểm phù hợp với sở thích cá nhân còn gặp nhiều khó khăn do thiếu hệ thống gợi ý hiệu quả. Người dùng thường phải tự tìm kiếm và đánh giá thông tin từ nhiều nguồn khác nhau.

Từ thực tế đó, TourXport được đề xuất nhằm xây dựng một hệ thống hỗ trợ du lịch thông minh, giúp:

- Cá nhân hóa việc gợi ý địa điểm
- Tích hợp bản đồ và định vị
- Xây dựng lộ trình di chuyển tối ưu

Hệ thống giải quyết các yêu cầu chính:

- Gợi ý địa điểm dựa trên nhu cầu người dùng.

- Hiển thị thông tin điểm đến.
- Xác định vị trí người dùng.
- Tạo đường đi giữa các địa điểm.
- Cung cấp giao diện trực quan và dễ sử dụng.

3 Mục tiêu và phạm vi POC

3.1 Mục tiêu POC

POC (Proof of Concept) của dự án TourXport được xây dựng nhằm kiểm chứng tính khả thi của việc phát triển một ứng dụng hỗ trợ du lịch thông minh trên nền tảng di động.

Các mục tiêu chính bao gồm:

- Hiển thị danh sách gợi ý các địa điểm tham quan.
- Xây dựng giao diện trực quan, dễ thao tác với người dùng.
- Khảo sát nhu cầu người dùng.
- Xây dựng chức năng tạo tuyến đường giữa các địa điểm.

3.2 Phạm vi POC

Trong giai đoạn POC, nhóm tập trung phát triển các chức năng cốt lõi nhằm chứng minh tính khả thi của hệ thống, bao gồm:

- Chức năng thực hiện:
 - Đăng ký, đăng nhập
 - Hiển thị danh sách gợi ý các địa điểm tham quan.
 - Lưu danh sách những địa điểm đã thích
 - Khảo sát nhu cầu người dùng
 - Tạo và hiển thị tuyến đường di chuyển.

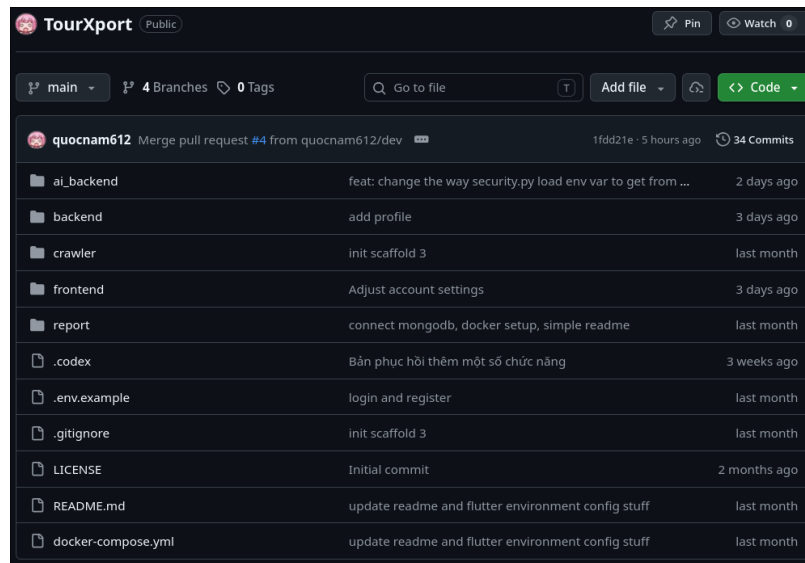
- Giao diện trực quan, dễ thao tác với người dùng.
- Ngoài phạm vi POC:
 - Hệ thống AI gợi ý địa điểm theo nhu cầu người dùng.
 - Tối ưu hiệu năng cho quy mô lớn.
 - Bảo mật thông tin người dùng.

CHƯƠNG 2: KIẾN TRÚC HỆ THỐNG TỔNG THỂ

1 Sơ đồ kiến trúc tổng quát

Hệ thống TourXport được thiết kế dựa trên kiến trúc client-server hiện đại, phân tách rõ ràng giữa các lớp xử lý nhằm đảm bảo tính mở rộng và dễ dàng bảo trì. Kiến trúc tổng quát bao gồm ba thành phần chính:

- **Lớp ứng dụng (Client-side):** Được xây dựng bằng framework Flutter, đóng vai trò là giao diện tương tác trực tiếp với người dùng. Thành phần này chịu trách nhiệm thu thập yêu cầu, hiển thị dữ liệu và quản lý trạng thái ứng dụng.
- **Lớp dịch vụ (Server-side):** Sử dụng môi trường Node.js [5] và framework Express [6]. Lớp này đóng vai trò trung tâm điều phối, xử lý các logic nghiệp vụ, xác thực người dùng và giao tiếp với các dịch vụ hỗ trợ.
- **Lớp dịch vụ AI (AI Backend):** Một hệ thống riêng biệt chuyên trách xử lý các yêu cầu liên quan đến mô hình ngôn ngữ lớn (LLM) và kỹ thuật RAG (Retrieval-Augmented Generation) để cung cấp các gợi ý hành trình cá nhân hóa.
- **Lớp lưu trữ (Database):** Sử dụng MongoDB [7] để lưu trữ dữ liệu dưới dạng tài liệu (Document-oriented), cho phép cấu trúc dữ liệu linh hoạt phù hợp với các thông tin du lịch đa dạng.

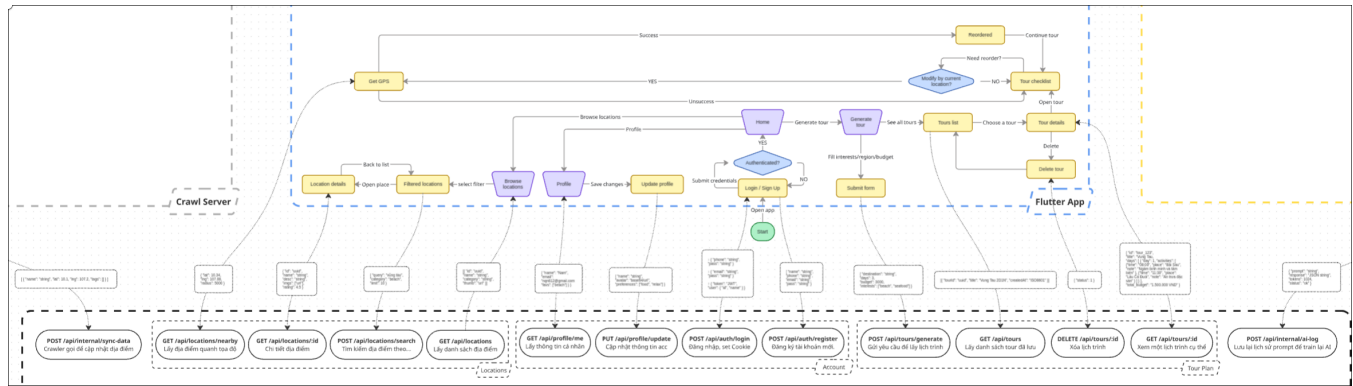


Hình 2.1: Kiến trúc thư mục của dự án TourXport

2 Luồng giao tiếp giữa các thành phần

Luồng dữ liệu trong hệ thống được thực hiện thông qua giao thức HTTP/HTTPS với định dạng dữ liệu chuẩn JSON. Quy trình tương tác diễn hình được thực hiện qua các bước sau:

1. **Gửi yêu cầu:** Client gửi yêu cầu kèm theo mã xác thực JWT (JSON Web Token) đến các điểm cuối (endpoints) tương ứng trên Backend Server.
2. **Xử lý tại Backend:** Server thực hiện kiểm tra quyền truy cập thông qua các Middleware. Tùy theo loại yêu cầu, server sẽ truy vấn trực tiếp vào cơ sở dữ liệu hoặc gửi tiếp yêu cầu đến dịch vụ AI.
3. **Xử lý AI và Dữ liệu:** Dịch vụ AI tiếp nhận yêu cầu, thực hiện truy xuất dữ liệu từ các nguồn đã được chuẩn hóa (Scraped data) để tạo ra phản hồi thông minh.
4. **Phản hồi:** Kết quả được đóng gói và trả về Client. Lớp ứng dụng thực hiện giải mã JSON và cập nhật giao diện người dùng ngay lập tức.



Hình 2.2: Sơ đồ luồng giao tiếp giữa các thành phần của hệ thống TourXport

3 Thiết kế Cơ sở dữ liệu tập trung trên MongoDB

Cơ sở dữ liệu được thiết kế theo mô hình tập trung, tận dụng khả năng lưu trữ không cấu trúc của MongoDB để tối ưu hóa tốc độ truy xuất. Các thực thể chính bao gồm:

3.1 Cấu trúc người dùng (User Collection)

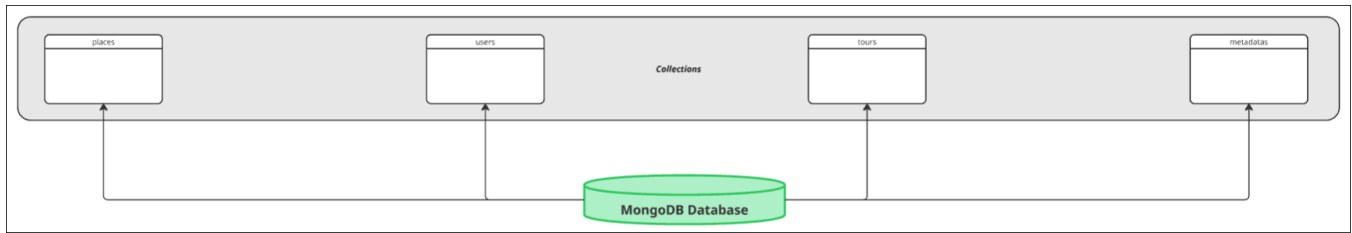
Lưu trữ thông tin định danh, thông tin cá nhân và các cài đặt bảo mật. Mật khẩu được mã hóa trước khi lưu trữ để đảm bảo an toàn dữ liệu.

3.2 Cấu trúc hành trình (Tour Collection)

Đây là thành phần cốt lõi của hệ thống, lưu trữ các Tour được tạo ra bởi AI. Cấu trúc bao gồm các mảng lồng nhau (Nested Arrays) để biểu diễn lịch trình theo từng ngày, bao gồm địa điểm, thời gian dự kiến và chi phí ước tính.

3.3 Cấu trúc địa điểm (Place Collection)

Chứa dữ liệu đã qua xử lý được thu thập bởi Data Engineer, bao gồm tọa độ địa lý, mô tả chi tiết, hình ảnh và đánh giá chất lượng. Việc tập trung hóa dữ liệu địa điểm giúp giảm độ trễ khi AI thực hiện truy xuất thông tin (RAG).



Hình 2.3: Sơ đồ thiết kế cơ sở dữ liệu trên MongoDB

CHƯƠNG 3: CHI TIẾT CÁC THÀNH PHẦN CỐT LÕI

1 Backend (Node.js/Express & Logic Engine)

1.1 Vai trò và Nhiệm vụ

Trong hệ thống TourXport, thành phần Backend đóng vai trò là "xương sống" kỹ thuật, chịu trách nhiệm vận hành toàn bộ các logic nghiệp vụ phức tạp và đảm bảo sự kết nối thông suốt giữa lớp ứng dụng (Frontend) và các dịch vụ dữ liệu/AI theo kiến trúc microservices [8]. Các nhiệm vụ chính bao gồm:

- **Thiết kế và quản lý API:** Xây dựng hệ thống RESTful API [9] cung cấp các điểm cuối (endpoints) cho ứng dụng di động truy xuất dữ liệu địa điểm, người dùng và hành trình.
- **Quản lý Cơ sở dữ liệu:** Thực hiện các thao tác CRUD (Create, Read, Update, Delete) trên MongoDB thông qua Mongoose ODM [10], đảm bảo tính toàn vẹn và tối ưu hóa truy vấn.
- **Xác thực và Bảo mật:** Triển khai cơ chế bảo mật dựa trên JSON Web Token (JWT) để quản lý phiên đăng nhập và bảo vệ các tài nguyên nhạy cảm của người dùng.
- **Tích hợp dịch vụ AI:** Đóng vai trò trung gian tiếp nhận yêu cầu từ người dùng, gửi dữ liệu ngữ cảnh (Context) cho dịch vụ AI và xử lý kết quả trả về trước khi lưu trữ.

1.2 Tech Stack & Thư viện

Hệ thống Backend được xây dựng trên các nền tảng công nghệ hiện đại, ưu tiên tính phi tập trung và hiệu năng xử lý bất đồng bộ:

- **Runtime Environment:** Node.js - cho phép xử lý số lượng lớn các kết nối đồng thời với mô hình Non-blocking Input/Output.
- **Web Framework:** Express.js - tối giản, linh hoạt, hỗ trợ mạnh mẽ việc xây dựng các Middleware xử lý lỗi và xác thực.
- **Database ODM:** Mongoose [10] - hỗ trợ định nghĩa Schema chặt chẽ cho MongoDB và quản lý các mối quan hệ dữ liệu phức tạp.

- **Bảo mật:** `bcryptjs` [11] (mã hóa mật khẩu) và `jsonwebtoken` (xác thực người dùng).
- **Quản lý môi trường:** Docker & Docker Compose - đóng gói toàn bộ dịch vụ để đảm bảo tính nhất quán giữa các môi trường phát triển và triển khai.

1.3 Quy trình xử lý

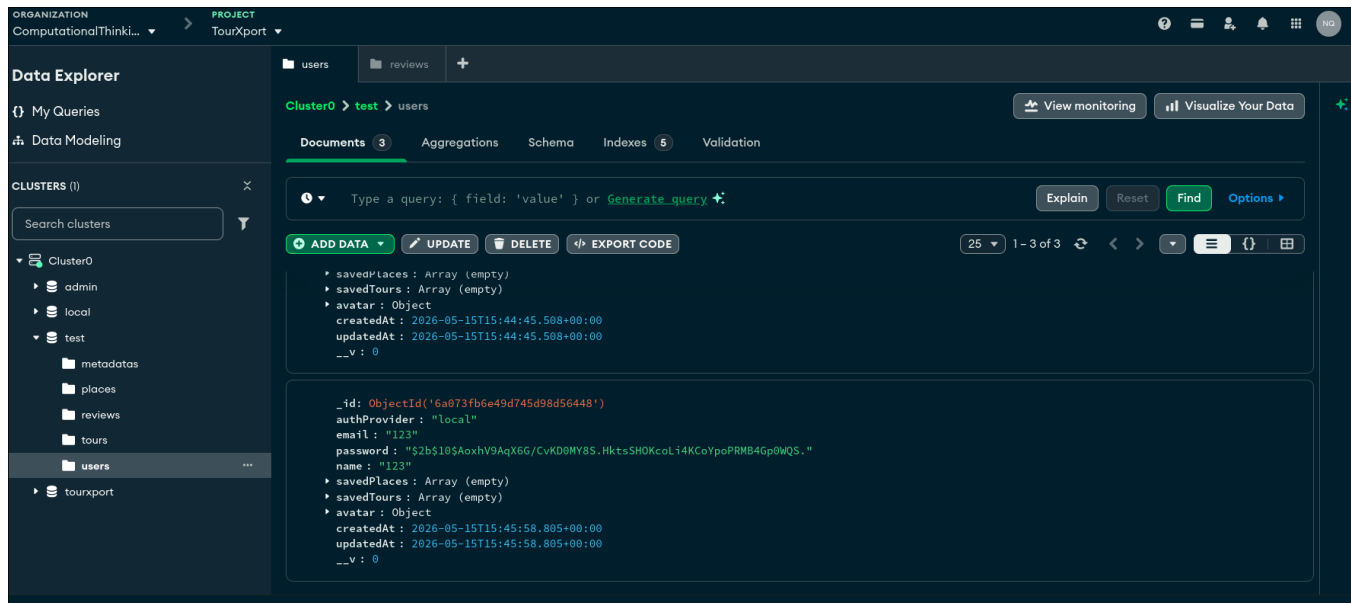
Quy trình xử lý một yêu cầu điển hình tại Backend được thực hiện qua các giai đoạn nghiêm ngặt:

1. **Tiếp nhận và Phân loại (Routing):** Yêu cầu từ Client được định tuyến đến Controller tương ứng dựa trên URL và phương thức HTTP (GET, POST, PUT, DELETE, PATCH).
2. **Kiểm soát truy cập (Middleware):** Các yêu cầu được đưa qua lớp kiểm tra Token. Nếu hợp lệ, thông tin người dùng được trích xuất từ JWT để sử dụng cho các bước tiếp theo.
3. **Thực thi Logic nghiệp vụ:** Controller thực hiện các tính toán hoặc gọi đến các Model để truy xuất dữ liệu. Trong trường hợp tạo tour, Backend sẽ giao tiếp với AI Backend để nhận kết quả lịch trình.
4. **Quản lý lỗi tập trung (Global Error Handling):** Mọi lỗi phát sinh trong quá trình xử lý đều được bắt bởi một Middleware chuyên trách, đảm bảo server không bị crash và trả về phản hồi chuẩn JSON cho người dùng.
5. **Phản hồi kết quả:** Kết quả sau khi xử lý được trả về dưới dạng JSON với cấu trúc chuẩn bao gồm trạng thái (*success*), thông điệp (*message*) và dữ liệu (*data*).

1.4 Kết quả đạt được

Sau quá trình triển khai và thực nghiệm trong giai đoạn POC, hệ thống Backend đã đạt được các kết quả cụ thể, đáp ứng đầy đủ các tiêu chí kỹ thuật đề ra:

- **Thiết lập thành công hệ thống Cơ sở dữ liệu Cloud:** Hệ thống đã triển khai hoàn thiện cơ sở dữ liệu NoSQL trên nền tảng MongoDB Atlas. Các Collection cốt lõi bao gồm `users`, `places`, `reviews` và `tours` đã được định nghĩa Schema chặt chẽ, sẵn sàng cho việc lưu trữ và truy xuất dữ liệu quy mô lớn.



Hình 3.1: Giao diện quản lý MongoDB Atlas với các Collection đã thiết lập

- **Xây dựng bộ thư viện API đầy đủ (API Documentation):** Toàn bộ các luồng nghiệp vụ quan trọng đã được hiện thực hóa qua hệ thống RESTful API. Nhóm đã xây dựng tài liệu theo dõi tiến độ chi tiết bao gồm các phương thức (GET, POST), điểm cuối (Endpoints), cấu trúc Request/Response và các trường hợp xử lý lỗi (Error cases). Các chức năng đã hoàn thiện gồm:
 - Hệ thống xác thực: Đăng ký (/auth/register), Đăng nhập (/auth/login).
 - Quản lý dữ liệu: Truy xuất danh sách địa điểm (/locations), chi tiết địa điểm và quản lý hồ sơ người dùng.

Hình 3.2: Bảng các Endpoints và trạng thái hoàn thiện của API

- The screenshot shows the Postman interface with a POST request to `http://localhost:3000/auth/login`. The request body is JSON: `{ "email": "123", "password": "123" }`. The response status is 200 OK, and the response body is JSON: `{ "success": true, "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZC6IjzhMDczNmZlZTQ5ZDc0NWQ5OGQ1NjQ0CiwiImShbnVlOikxMjMiLCJlbWFnbiCI6IjEyMyIsImhhbmQiOiJmcmVudC9yYXNpdFwufu1fVkABTngGLmZwlrlQ8hbWQQwpAzo", "user": { "id": "6a073fb6e49d745d98d56448", "name": "123", "avatar": "" } }`.

- **Đảm bảo tính toàn vẹn và bảo mật dữ liệu:** Mật khẩu người dùng đã được mã hóa dưới dạng Hash (thuật toán Bcrypt) trước khi lưu vào cơ sở dữ liệu, đảm bảo ngay cả quản

trị viên hệ thống cũng không thể truy cập mật khẩu gốc, đáp ứng tiêu chuẩn bảo mật trong phát triển phần mềm hiện đại.

2 Frontend (Flutter Multi-platform)

2.1 Vai trò và Nhiệm vụ

Frontend đóng vai trò lớp giao tiếp trực tiếp giữa người dùng và hệ thống TourXport, chịu trách nhiệm xây dựng giao diện, xử lý tương tác và hiển thị dữ liệu một cách trực quan.

Trong dự án TourXport, frontend được phát triển bằng Flutter nhằm hỗ trợ đa nền tảng và đảm bảo tính đồng nhất về giao diện trên các thiết bị di động.

Nhiệm vụ chính của frontend trong giai đoạn POC:

- Xây dựng Trải nghiệm Người dùng (UX)
- Phát triển Giao diện (UI)
- Thu thập Dữ liệu Người dùng
- Xử lý và Hiển thị Dữ liệu
- Quản lý Tài khoản

2.2 Tech Stack & Thư viện

Front-end của TourXport được xây dựng trên nền tảng công nghệ hiện đại nhằm đảm bảo hiệu năng và tính thẩm mỹ cao:

- **Framework chính:** Flutter cho phép xây dựng ứng dụng đa nền tảng (Android, iOS, Web, Windows) từ một codebase duy nhất.
- **Ngôn ngữ lập trình:** Dart là ngôn ngữ tối ưu cho việc xây dựng UI và các tác vụ bất đồng bộ.
- **Thư viện và Công cụ [12]:**
 - http: Xử lý các yêu cầu mạng (RESTful API) tới Backend.

- google_fonts: Tích hợp bộ font chữ Montserrat chuyên nghiệp
- **Công cụ hỗ trợ:** Figma - thiết kế UI/UX và prototype giao diện

2.3 Quy trình xử lý

Bước 1: Lên ý tưởng và phân tích yêu cầu

- Xác định chức năng cần phát triển
- Phân tích nhu cầu người dùng
- Xây dựng luồng sử dụng của ứng dụng

Bước 2: Thiết kế giao diện trên Figma

- Chọn màu sắc, font chữ, bố cục
- Tạo prototype mô phỏng trải nghiệm người dùng

Bước 3: Phát triển giao diện bằng Dart với Flutter

- Chuyển thiết kế thành giao diện thực tế
- Xây dựng widget và xử lý logic frontend
- Kết nối API và quản lý trạng thái ứng dụng

Bước 4: Kiểm thử trên máy ảo

- Chạy thử trên Android Emulator
- Kiểm tra responsive và hiệu năng
- Phát hiện và sửa lỗi giao diện/chức năng

2.4 Phân chia công việc

2.4.1 Sprint 1

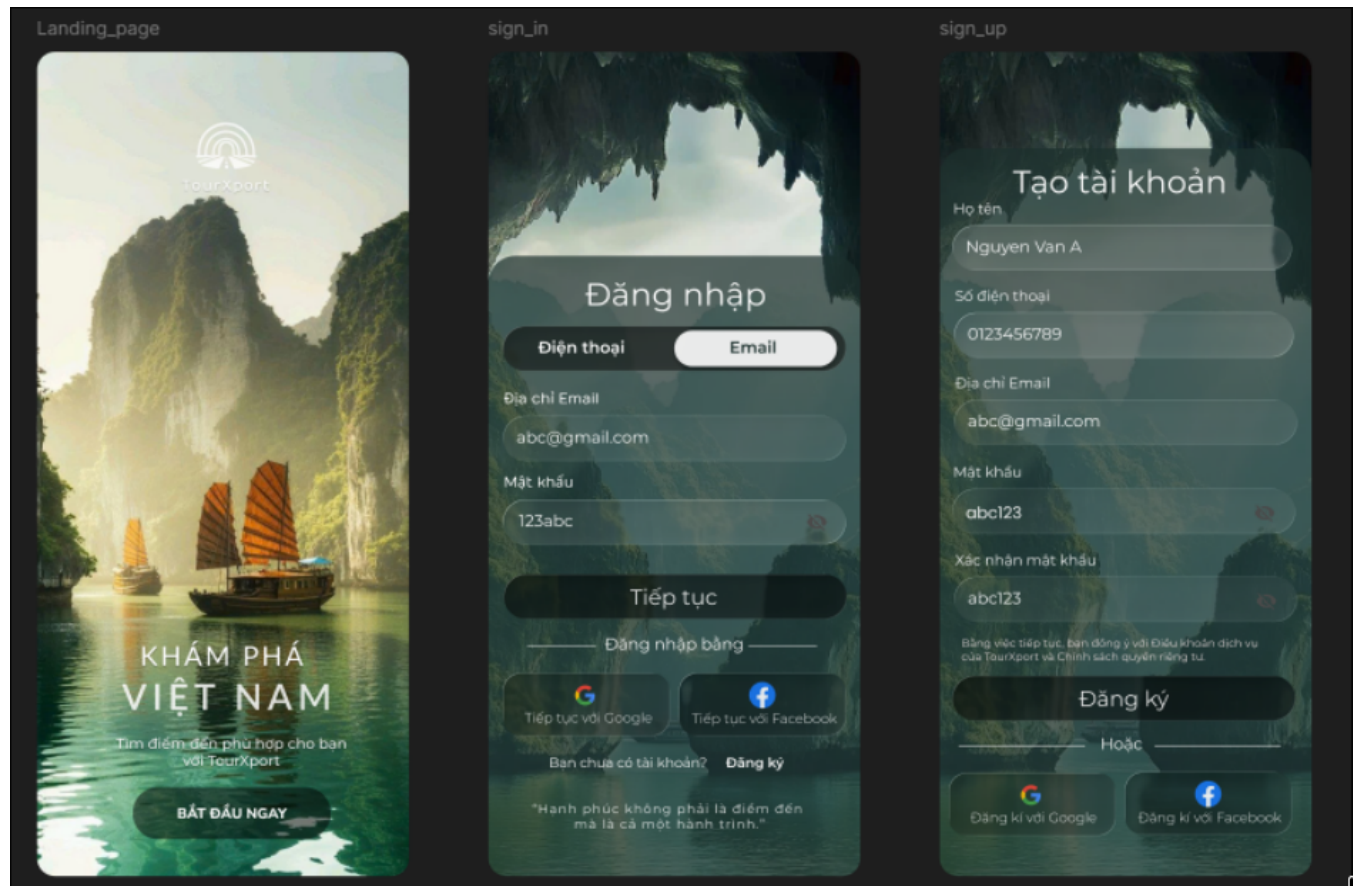
Front-end									
set up các techstack cần thiết		A, B							set up thành công Flutter, Android Studio
lên ý tưởng thiết kế giao diện			A						trình bày ý tưởng thiết kế với các thành viên
design landing page				A					landing page trong figma
design sign-in page					B	B			sign-in page trong figma
design sign-up page							A		sign-up page trong figma

Hình 3.4: Bảng theo dõi công việc front-end sprint 1

Trong Sprint 1, nhóm Front-end tập trung xây dựng nền tảng ban đầu cho ứng dụng bằng Flutter và ngôn ngữ Dart. Mục tiêu chính của sprint là hoàn thành môi trường phát triển, lên ý tưởng giao diện và thiết kế các màn hình cơ bản trên Figma để chuẩn bị cho giai đoạn lập trình.

Các công việc đã thực hiện gồm:

- Cài đặt và cấu hình các công nghệ cần thiết như Flutter, Dart và Android Studio.
- Thảo luận và thống nhất ý tưởng thiết kế giao diện với các thành viên trong nhóm.
- Thiết kế landing page, sign-in page và sign-up page trên Figma.
- Xây dựng cấu trúc dự án Flutter ban đầu và kiểm tra khả năng chạy ứng dụng trên máy ảo Android Emulator.



Hình 3.5: Thiết kế landing page, sign in, sign up

Kết quả đạt được sau Sprint 1:

- Môi trường phát triển Front-end đã được thiết lập hoàn chỉnh.
- Bộ giao diện cơ bản đã được thiết kế thống nhất.
- Ứng dụng Flutter có thể khởi chạy và chạy thử nghiệm thành công trên máy ảo.

2.4.2 Sprint 2

Front-end									
code screen dashboard				A					chạy được trên emulator, sau khi đăng nhập
code screen place					B				sau khi chọn nơi đó
design screen survey					A				screen figma khảo sát người dùng
design screen save						B			screen danh sách những địa điểm người dùng lưu

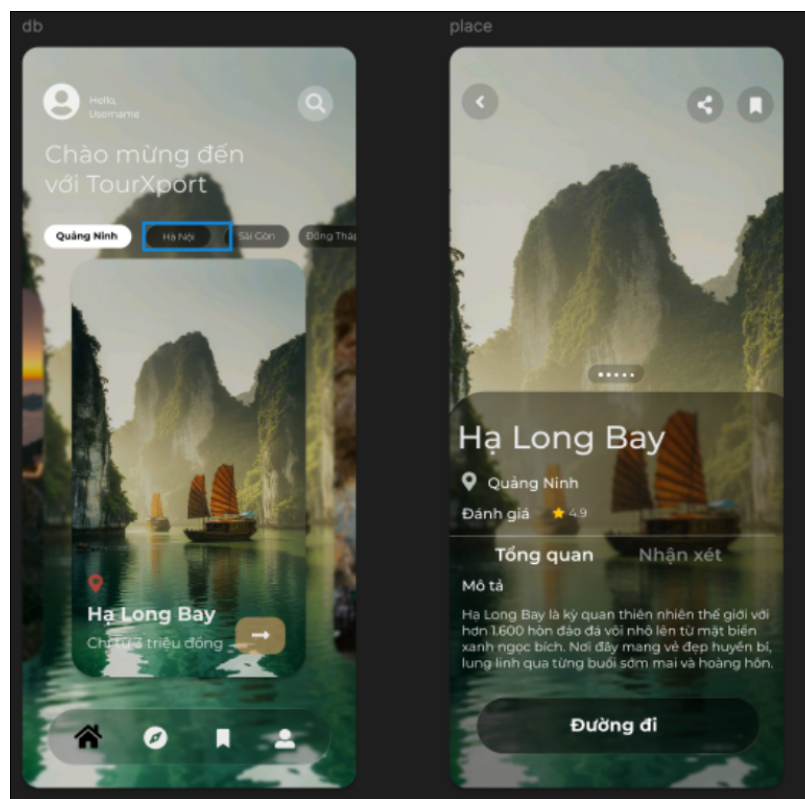
Hình 3.6: Bảng theo dõi công việc front-end sprint 2

Trong Sprint 2, nhóm tiếp tục phát triển các chức năng chính của ứng dụng bằng Flutter và Dart. Sprint này tập trung vào việc xây dựng các màn hình chức năng sau khi người dùng đăng

nhập, đồng thời mở rộng thiết kế giao diện phục vụ trải nghiệm cá nhân hóa.

Các công việc đã thực hiện gồm:

- Phát triển màn hình Dashboard và tích hợp điều hướng cơ bản sau khi đăng nhập.
- Xây dựng màn hình Place để hiển thị danh sách địa điểm và chức năng chọn nơi đến.
- Thiết kế màn hình Survey nhằm khảo sát sở thích và nhu cầu của người dùng.
- Thiết kế màn hình Save để hiển thị danh sách các địa điểm đã được lưu.



Hình 3.7: Thiết kế dashboard và places

Kết quả đạt được sau Sprint 2:

- Ứng dụng có thể điều hướng giữa các màn hình chính trên Android Emulator.
- Các chức năng giao diện cơ bản sau đăng nhập đã được xây dựng thành công.
- Hệ thống giao diện tiếp tục được hoàn thiện và đồng bộ theo thiết kế trên Figma.

2.4.3 Sprint 3

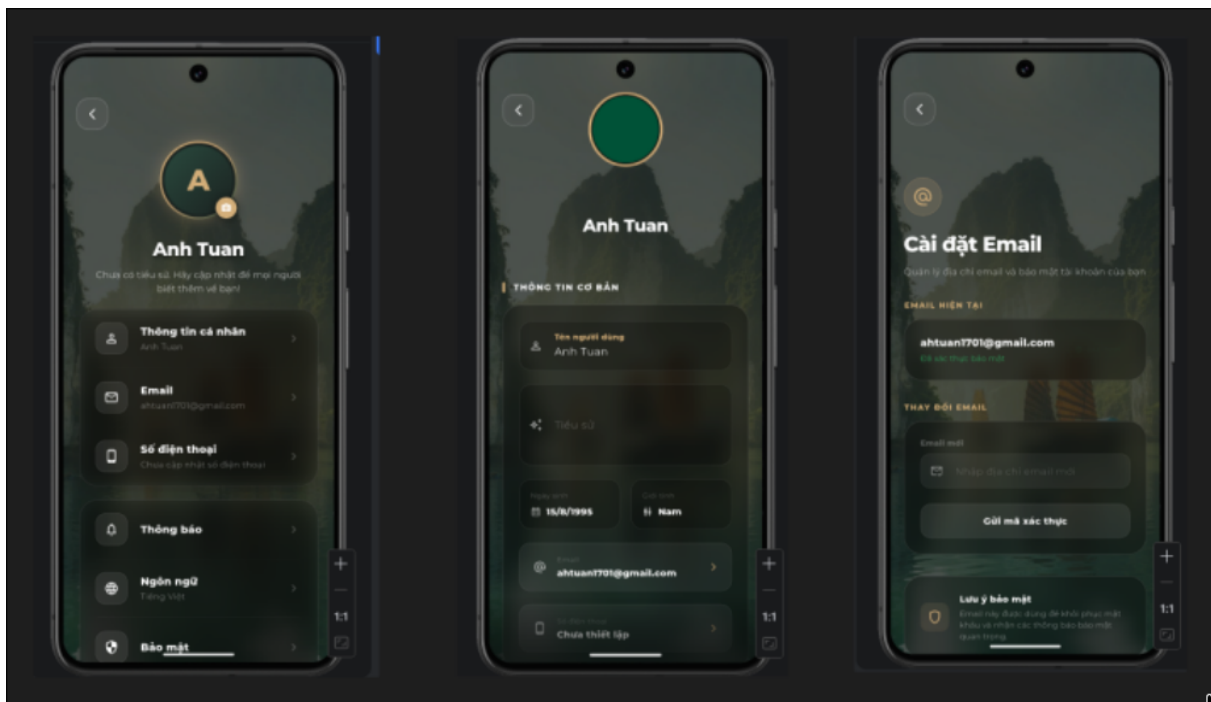
Front-end									
screen profile				A					giao diện phù hợp, chạy được trên máy ảo
screen thông tin cá nhân, gmail, sdt					B				giao diện phù hợp, chạy được trên máy ảo
screen thông báo, ngôn ngữ, hỗ trợ					A	B			giao diện phù hợp, chạy được trên máy ảo

Hình 3.8: Bảng theo dõi công việc front-end sprint 3

Trong Sprint 3, tiếp tục hoàn thiện các chức năng liên quan đến tài khoản và trải nghiệm người dùng bằng Flutter và Dart. Sprint này tập trung phát triển các màn hình cá nhân hóa và cài đặt trong ứng dụng.

Các công việc đã thực hiện gồm:

- Xây dựng màn hình Profile với giao diện phù hợp và khả năng hoạt động trên máy ảo.
- Phát triển màn hình thông tin cá nhân bao gồm họ tên, Gmail và số điện thoại.
- Thiết kế và hoàn thiện màn hình Thông báo, Ngôn ngữ và Hỗ trợ.
- Tối ưu bố cục giao diện nhằm đảm bảo tính đồng bộ và thân thiện với người dùng.



Hình 3.9: Màn hình máy ảo profile và trang thông tin cá nhân, email

Kết quả đạt được sau Sprint 3:

- Các màn hình quản lý tài khoản và cài đặt đã được xây dựng thành công.
- Ứng dụng tiếp tục hoạt động ổn định trên Android Emulator.
- Giao diện người dùng được hoàn thiện hơn về tính trực quan và trải nghiệm sử dụng.

3 AI Worker (LLM & RAG Engine)

3.1 Vai trò và Nhiệm vụ

AI Worker là thành phần xử lý trí tuệ nhân tạo cốt lõi của hệ thống TourXport, chịu trách nhiệm diễn giải các yêu cầu du lịch phức tạp của người dùng (thông qua văn bản tự nhiên, ngân sách, thời gian) và tự động lập kế hoạch lịch trình chi tiết (Itinerary). Việc tách biệt AI Worker thành một dịch vụ độc lập (Microservice) khỏi Backend chính giúp tối ưu hóa việc xử lý các tác vụ tốn kém thời gian và tài nguyên tính toán, đặc biệt là quá trình giao tiếp mạng bất đồng bộ với các mô hình ngôn ngữ lớn (LLMs).

Thành phần này đóng vai trò như một bộ điều phối (Orchestrator), quyết định việc sinh lịch trình tự do bằng LLM thuần túy (Pure LLM) hoặc kết hợp với công nghệ truy xuất dữ liệu (RAG - Retrieval-Augmented Generation) từ cơ sở dữ liệu vector. Trong tương lai, nếu triển khai các tính năng khác cần sử dụng LLM làm cốt lõi, module AI Worker cũng sẽ đóng vai trò là điều phối.

3.2 Tech Stack & Thư viện

- **Ngôn ngữ nền tảng:** Python 3.10.
- **Framework:** FastAPI (phiên bản 0.136.1) [13] – cung cấp khả năng xử lý bất đồng bộ (async) với hiệu suất cao và tự động hóa sinh tài liệu giao diện API.
- **LLM Integration:** Thư viện openai (phiên bản 2.36.0) cho python [14] – giao tiếp trực tiếp với mô hình gpt-4o-mini. Hệ thống sử dụng phương thức phân tích cú pháp nâng cao để đảm bảo đầu ra luôn tuân thủ nghiêm ngặt chuẩn cấu trúc JSON.
- **Data Validation & Serialization:** Thư viện pydantic (phiên bản 2.13.4) [15] – định nghĩa các schema ràng buộc kiểu dữ liệu cho Request và Response. Hệ thống tận dụng Pydantic để

tự động ánh xạ cấu trúc dữ liệu snake_case của Python sang định dạng camelCase chuẩn hóa cho luồng giao tiếp với Frontend.

- **Authentication:** Thư viện PyJWT [16] – dùng để giải mã và xác thực token JWT được cấp phát chéo từ Backend chính.

3.3 Quy trình xử lý

- **Bước 1:** Tiếp nhận và Xác thực Yêu cầu — Lớp Controller tại file `src/api/endpoints/trip_generator.py` (hàm `generate_trip`) tiếp nhận POST request chứa dữ liệu `TripGenerateRequest`. Tại đây, hệ thống thực thi hàm `dependency verify_token` (được định nghĩa trong `src/core/security.py`) để tiến hành giải mã chuỗi JWT truyền vào thông qua Header, nhằm xác minh danh tính và phiên thao tác của người dùng.
- **Bước 2:** Điều phối Luồng xử lý (Orchestrator) — Sau khi xác thực, dữ liệu được chuyển cho lớp `TripOrchestrator` tại `src/services/trip_generator/ai_orchestrator.py` (hàm `execute`). Trong phạm vi giai đoạn thực thi POC hiện tại, hàm này quyết định chuyển hướng luồng xử lý thẳng đến module sinh kế hoạch bằng LLM thuần túy (`generate_itinerary_pure_llm`), do hệ thống cơ sở dữ liệu địa điểm (Vector Database) vẫn đang trong quá trình xây dựng và cào dữ liệu. Các logic tương lai dành cho RAG (như Vector Search qua `DBService`) đã được thiết kế sẵn cấu trúc (placeholder) nhằm chờ tích hợp.
- **Bước 3:** Giao tiếp LLM với Structured Output — Lớp `AIService` tại `src/services/trip_generator/ai_service.py` tiếp nhận tham số đầu vào (ngân sách, số ngày, sở thích) để cấu trúc thành System Prompt và User Prompt. Hệ thống gọi hàm API của OpenAI thông qua phương thức `beta.chat.completions.parse`, đồng thời truyền vào tham số `response_format=TripGeneratorResponse`. Kỹ thuật này ép buộc mô hình gpt-4o-mini không trả về văn bản tự do, mà phải trả về chính xác cấu trúc dữ liệu định nghĩa tại `src/schemas/response/trip_generator.py` (bao gồm các thực thể lồng nhau như `TripData`, `DailyItinerary`, và `Location`).
- **Bước 4:** Chuẩn hóa và Phản hồi — Kết quả JSON trả về từ OpenAI lập tức được Pydantic parse thành đối tượng Python nội bộ. Hệ thống chủ động gán cờ `is_from_db = False` để minh bạch hóa với Frontend rằng dữ liệu này hoàn toàn do AI khởi tạo (ảo hóa), các thuộc

tính như `place_id` sẽ nhận giá trị null. Cuối cùng, FastAPI tự động serialize đối tượng này và phản hồi về cho phía Client (Flutter).

4 Thu thập Dataset (Data Crawler/Scraper)

4.1 Vai trò và Nhiệm vụ

4.1.1 Vai trò

Module Crawler/Scraper đóng vai trò là lớp thu thập dữ liệu đầu tiên của hệ thống. Thay vì cào dữ liệu HTML trực tiếp, nhóm sử dụng RapidAPI để bảo đảm ổn định, hợp pháp và có cấu trúc dữ liệu rõ ràng.

Vai trò chính của module này gồm:

- **Thu thập dữ liệu:** Tự động hóa việc truy xuất thông tin về địa điểm (khách sạn, nhà hàng, điểm du lịch), các hình ảnh minh họa và các bài đánh giá (reviews) của người dùng từ hệ thống TripAdvisor.
- **Tiền xử lý dữ liệu:** Chuẩn hóa và lưu trữ dữ liệu thu thập được dưới định dạng JSON theo định dạng mong muốn, sẵn sàng cho việc phân tích và huấn luyện mô hình AI trong tương lai.

4.1.2 Nhiệm vụ

Nhiệm vụ chính trong giai đoạn POC gồm:

- Kết nối và tích hợp API key của RapidAPI để gọi các endpoint của Travel Advisor như `locations/search` và `photos/list`.
- Lấy các trường dữ liệu cốt lõi như tên địa điểm, ID địa điểm, điểm đánh giá, nội dung review và ảnh minh họa.
- Sử dụng các tham số đầu vào của endpoint để lấy đủ số lượng địa điểm, ảnh và đánh giá thay vì chỉ lấy trang đầu tiên.
- Xử lý và tối ưu gọi RapidAPI bằng cách giới hạn lưu lượng requests (x requests/ giây, y requests/ tháng) để tránh lỗi HTTP 429 (Too Many Requests) và tránh lãng phí tokens RapidAPI.

- Chuyển đổi dữ liệu thô thành dữ liệu JSON tối ưu và lưu trữ cục bộ.

4.2 Tech Stack & Thư viện

- **Ngôn ngữ lập trình:** Python 3.9+.
- **Thư viện chính:**
 - requests: Thực hiện các HTTP GET requests đến endpoint của RapidAPI
 - json: phân tích dữ liệu trả về từ API.
 - time: Tạo độ trễ giữa các lần gọi API, tuân thủ Rate Limit của gói Rapid API đang sử dụng.
 - python-dataenv: Quản lý biến môi trường, bảo mật thông tin X-RapidAPI-Key và X-RapidAPI-Host thay vì hard-code trực tiếp vào mã nguồn.

4.3 Quy trình xử lý

- **Bước 1: Khởi tạo & Xác thực**
 - Load cấu hình từ file .env (API key, Host URL)
 - Thiết lập HTTP Headers theo yêu cầu của RapidAPI (bao gồm X-RapidAPI-Key và X-RapidAPI-Host)
- **Bước 2: Tìm kiếm ID địa điểm.**
 - Gọi endpoint locations/search sử dụng biến số từ khóa địa điểm để lấy dữ liệu trường location_id.
 - Trích xuất và lưu lại location_id của địa điểm cần lấy data
- **Bước 3: Thu thập thông tin điểm du lịch.**
 - Sử dụng location_id làm biến số gọi endpoint attraction/list, thu thập thông tin các điểm du lịch lân cận, truy xuất lấy location_id của từng điểm
 - Sử dụng location_id của từng điểm gọi endpoint attraction/get-details để lấy thông tin chi tiết

- Lọc qua cấu trúc file JSON dữ lại những trường cốt lõi, bỏ những trường dư thừa không sử dụng.
 - Cộng dồn offset để quản lý danh sách theo trang.
 - Trong quá trình hệ thống vận hành, kiểm tra HTTP Status Code (Nếu 200 OK -> đi tiếp; Nếu 429 -> Tạm dừng hệ thống 10-30 giây và retry; Nếu lỗi khác -> Ghi log và bỏ qua)
 - Kích hoạt Rate Limiting Rule với time.sleep trước khi gọi trang tiếp theo.
- **Bước 4: Chuẩn hóa & Lưu trữ.**
 - Ép kiểu các trường thời gian mở và đóng cửa thành dạng giờ trong ngày cũng như nhiều trường khác
 - Lưu Data thành file .json để bắt đầu phân tích và huấn luyện mô hình

CHƯƠNG 4: THỰC THI VÀ ĐÁNH GIÁ THỰC NGHIỆM

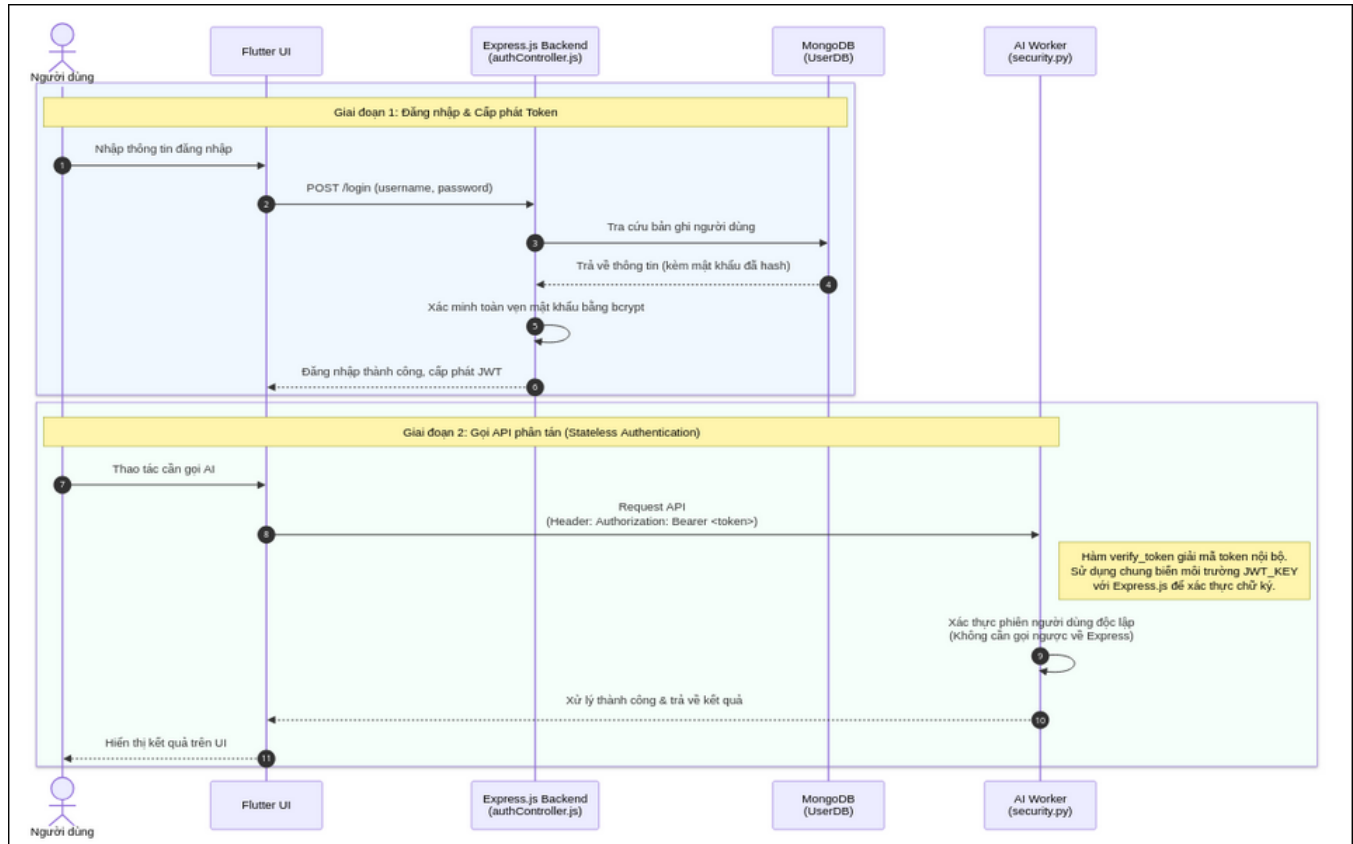
1 Xây dựng môi trường và triển khai

TourXport được đóng gói trong các container thông qua công nghệ Docker Compose [17], đảm bảo tính nhất quán tuyệt đối về cấu hình môi trường từ máy phát triển cục bộ đến máy chủ thực tế. Tập lệnh docker-compose.yml định nghĩa hạ tầng triển khai song song ba dịch vụ độc lập:

- **Dịch vụ backend:** Triển khai trên môi trường Node.js (Express.js) [5, 6] (cổng 3000), thực hiện volume mapping với mã nguồn tại thư mục `./backend` và sử dụng công cụ nodemon để tự động nạp lại khi có sự thay đổi mã nguồn.
- **Dịch vụ ai_backend (AI Worker):** Triển khai môi trường Python 3.10-slim (cổng 8000). Tập lệnh Dockerfile tự động hóa quá trình cài đặt phụ thuộc từ requirements.txt và khởi chạy máy chủ Uvicorn. Dịch vụ này cũng áp dụng cơ chế watch mode để đồng bộ tự động mã nguồn từ thư mục `src/` vào bộ chứa. ai_backend chỉ được expose qua port 8000 nội bộ của Docker, nên chỉ mỗi backend Node.js mới giao tiếp được với ai_backend.
- **Frontend (Flutter):** Được biên dịch và khởi chạy trực tiếp trên thiết bị thực tế hoặc máy ảo thông qua công cụ dòng lệnh flutter run. Ứng dụng client sẽ giao tiếp với cụm máy chủ thông qua các kiến trúc RESTful API.

2 Kết quả thực thi các Use Case

2.1 Use Case Đăng nhập và Xác thực



Hình 4.1: Sequence Diagram mô tả use case Đăng nhập và xác thực

Khi người dùng gửi thông tin đăng nhập qua giao diện Flutter, Backend Express.js (tại authController.js, hàm login) sẽ tiến hành tra cứu bản ghi trong tập hợp UserDB và xác minh tính toàn vẹn của mật khẩu bằng thuật toán mã hóa bcrypt. Khi đăng nhập thành công, hệ thống cấp phát một JSON Web Token (JWT).

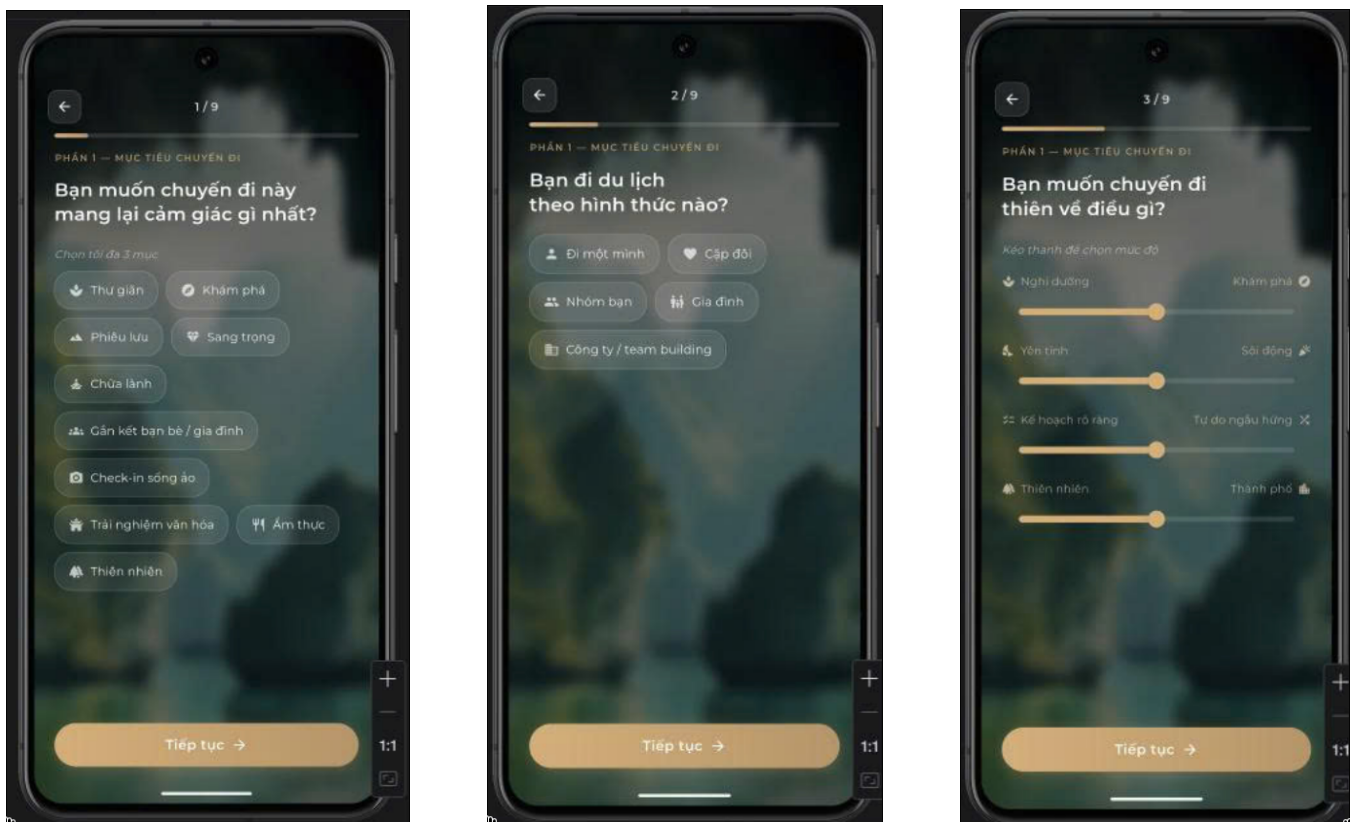
Token này sau đó được gắn vào header của giao thức HTTP (Authorization: Bearer <token>) cho mọi yêu cầu (request) về sau. Điểm đáng chú ý trong kiến trúc phân tán là khi người dùng gọi API của AI Worker, hệ thống AI tự động áp dụng hàm verify_token (ai_backend/src/core/security.py) để giải mã token nội bộ. Do AI Worker sử dụng chung một khóa bí mật (biến môi trường JWT_KEY) với Backend Express.js, nó có thể xác thực được phiên người dùng mà không cần phải gọi API ngược về Backend Express.js, giúp giảm chi phí độ trễ (latency).

2.2 Use Case Khảo sát nhu cầu người dùng (Survey)

Trong giai đoạn POC hiện tại, quy trình của Use Case này đang tập trung vào việc xây dựng trải nghiệm thu thập thông tin người dùng ở phía Frontend. Cụ thể, ứng dụng Flutter đã hoàn thiện thiết kế giao diện khảo sát (Survey Screen), cho phép du khách nhập các thông số đầu vào cần thiết cho chuyến đi, bao gồm:

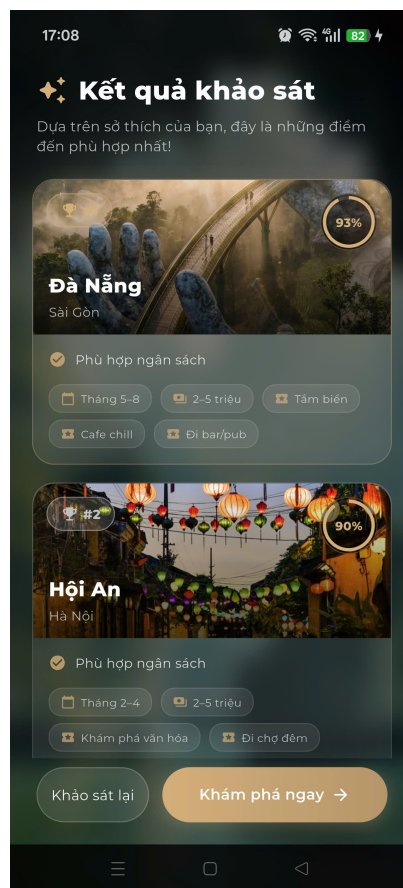
- Ngân sách tối đa.
- Số ngày dự kiến.
- Các sở thích cá nhân cụ thể.

Hiện tại, hệ thống đã hoàn tất phần thiết kế UI/UX và các logic quản lý trạng thái biểu mẫu (form state management) trên thiết bị người dùng. Bước tích hợp (Integration) để đóng gói dữ liệu này dưới định dạng JSON và thực hiện lời gọi API thực tế tới endpoint POST /api/trip/generate của AI Worker đang trong lộ trình phát triển và sẽ được hoàn thiện ở các bản cập nhật tiếp theo.



Hình 4.2: Màn hình câu hỏi Survey - Mô phỏng trên máy ảo Android

Tuy quy trình end-to-end từ Frontend đến Backend chưa được liên kết, toàn bộ luồng xử lý lỗi ở phía máy chủ (server-side) đã được xây dựng và kiểm thử độc lập thành công. Khi Frontend sẵn sàng gửi dữ liệu, AI Worker đã có sẵn cơ chế kiểm duyệt dữ liệu tự động thông qua mô hình Pydantic (TripGenerateRequest tại `ai_backend/src/schemas/request/trip_generator.py`). Các request vi phạm điều kiện logic, như ngân sách âm hoặc số ngày vượt quá ngưỡng 30, sẽ bị từ chối ngay lập tức tại cổng API kèm mã lỗi 422 Unprocessable Entity. Khi nhận dữ liệu hợp lệ, luồng Orchestrator sẽ kết nối với OpenAI để sinh ra danh sách các hoạt động du lịch (Location) được phân bổ theo ngày và buổi, tạo tiền đề để Frontend render giao diện timeline sau này.



Hình 4.3: Kết quả từ Survey - Mô phỏng trên máy ảo Android

2.3 Đánh giá độ chính xác và hiệu năng hệ thống

2.3.1 Hiệu năng hệ thống và độ trễ

Hệ thống thể hiện hiệu năng tổng thể ổn định và tối ưu xuyên suốt các tầng kiến trúc. Ở phía Client, Frontend Flutter vận hành mượt mà trên đa nền tảng nhờ engine kết xuất đồ họa hiệu năng

cao, quản lý trạng thái hiệu quả giúp UI không bị gián đoạn ngay cả khi hiển thị các dữ liệu phức tạp. Tại Server-side, Backend Node.js (Express) xử lý các tác vụ IO chuyên sâu, như truy vấn dữ liệu địa điểm và xác thực người dùng, với độ trễ cực thấp nhờ cơ chế non-blocking I/O.

Đối với hệ thống AI Worker (FastAPI), việc sử dụng hoàn toàn luồng xử lý bất đồng bộ (Asyncio kết hợp thư viện AsyncOpenAI và Motor) giúp giải quyết triệt để nút thắt cổ chai hệ thống. Hiện tại, độ trễ lớn nhất của hệ thống chủ yếu phụ thuộc vào thời gian xử lý tự nhiên của OpenAI API, dao động từ 2 đến 5 giây tùy vào độ dài prompt, trong khi các thành phần nội bộ đều phản hồi gần như tức thời.

2.3.2 Độ chính xác và tính toàn vẹn dữ liệu

Dữ liệu của dự án được bảo vệ và kiểm duyệt qua nhiều lớp. Trên Frontend, người dùng bị ràng buộc bởi các form validation, như nhập đúng định dạng email và ngân sách. Khi dữ liệu đến Backend Express, tính toàn vẹn tiếp tục được đảm bảo thông qua hệ thống Mongoose Schema, ví dụ UserDB ép buộc tính duy nhất của email và số điện thoại, bcrypt băm mật khẩu an toàn.

Đối với luồng sinh dữ liệu bằng trí tuệ nhân tạo, hệ thống khắc phục hoàn toàn điểm yếu phổ biến của LLM là trả về JSON bị hỏng (malformed data). Nhờ áp dụng tính năng Structured Outputs của OpenAI kết hợp với cơ chế validate và parse chặt chẽ của Pydantic, tỷ lệ lỗi format kết quả giảm xuống ngưỡng tiệm cận 0

2.3.3 Tính mô-đun và khả năng mở rộng

Việc thiết kế kiến trúc phân tán (Microservices-oriented) phân tách rạch ròi giữa UI (Flutter), Logic quản trị lõi (Express.js), AI Engine (FastAPI) và Cơ sở dữ liệu tập trung (MongoDB Atlas) mang lại khả năng mở rộng độc lập tuyệt vời. Khi phát sinh trường hợp lưu lượng người dùng yêu cầu sinh lịch trình tăng đột biến, quản trị viên chỉ cần cấp phát thêm tài nguyên riêng cho các container của dịch vụ AI Worker.

Quá trình này diễn ra hoàn toàn biệt lập, không gây ảnh hưởng đến phần lõi quản lý thông tin người dùng hay truy xuất dữ liệu đang vận hành trên cụm Node.js. Song song đó, việc chọn MongoDB làm kho lưu trữ tập trung giúp cả hai backend, Node.js và Python, có thể giao tiếp và truy xuất khối lượng lớn dữ liệu phi cấu trúc một cách linh hoạt, tạo bản lề vững chắc cho việc triển khai các hệ thống phức tạp hơn như RAG (Retrieval-Augmented Generation) trong tương lai.

CHƯƠNG 5: TỔNG KẾT VÀ HƯỚNG PHÁT TRIỂN

1 Tổng kết

Dự án TourXport đã hoàn thành giai đoạn Proof of Concept - POC với những kết quả khả quan, khẳng định tính khả thi của một hệ thống hỗ trợ du lịch thông minh. Các mục tiêu cốt lõi được tổng kết như sau:

- **Xây dựng nền tảng kỹ thuật vững chắc:** Hệ thống đã triển khai thành công mô hình Client-Server sử dụng Flutter và Node.js. Việc áp dụng Docker giúp quá trình thiết lập môi trường và triển khai sản phẩm trở nên nhất quán và chuyên nghiệp.
- **Hiện thực hóa các tính năng cốt lõi:**
 - **Hệ thống xác thực người dùng:** Đã xây dựng hoàn chỉnh chức năng đăng ký và đăng nhập với bảo mật mật khẩu bằng thuật toán Bcrypt, cấp phát JWT token cho phiên làm việc của người dùng.
 - **Giao diện di động đầy đủ:** Phát triển thành công ứng dụng Flutter với các màn hình chính bao gồm Landing page, Sign-in, Sign-up, Dashboard, Place listing, Survey, Favorites (Save), Profile và Settings. Toàn bộ giao diện được thiết kế theo Material Design 3 với hiệu ứng glassmorphism cho cảm giác hiện đại.
 - **Hệ thống API RESTful:** Xây dựng bộ API đầy đủ trên Express.js để hỗ trợ quản lý người dùng, địa điểm, bài viết (reviews), và hành trình, với các endpoint chuẩn (GET, POST, PUT, DELETE).
 - **Công cụ sinh lịch trình AI:** Phát triển AI Worker với FastAPI sử dụng OpenAI API để tự động tạo các lịch trình du lịch chi tiết dựa trên yêu cầu của người dùng, với khả năng kết hợp dữ liệu (RAG) từ cơ sở dữ liệu MongoDB.
 - **Triển khai với Docker:** Đóng gói toàn bộ hệ thống (Backend, AI Worker, Database) vào các container Docker, sử dụng Docker Compose để quản lý và triển khai một cách nhanh chóng trên các môi trường khác nhau.
- **Quản trị dữ liệu tập trung:** Cơ sở dữ liệu MongoDB được thiết kế tối ưu, lưu trữ dữ liệu địa điểm đã qua chuẩn hóa, hỗ trợ hiệu quả cho việc truy xuất thông tin nhanh chóng.

- **Tích hợp Trí tuệ nhân tạo:** Đã thử nghiệm thành công việc kết nối với các mô hình ngôn ngữ lớn (LLM) để xử lý dữ liệu và đưa ra gợi ý hành trình sơ bộ, đặt nền móng cho các tính năng cá nhân hóa chuyên sâu.

2 Hướng phát triển trong tương lai

Để TourXport trở thành một ứng dụng du lịch toàn diện, hướng phát triển tiếp theo sẽ tập trung vào việc mở rộng hệ sinh thái dịch vụ và nâng cao trải nghiệm người dùng:

2.1 Mở rộng hệ sinh thái dịch vụ (Dịch vụ tích hợp)

- **Tích hợp ẩm thực và lưu trú:** Mở rộng cơ sở dữ liệu để gợi ý món ăn đặc sản và khách sạn theo ngân sách. Hệ thống AI sẽ tự động lồng ghép lịch trình ăn uống và nghỉ ngơi vào Tour dựa trên vị trí địa lý của các điểm tham quan.
- **Tính năng quản lý đoàn (Group Travel):** Phát triển cơ chế tạo nhóm thông qua mã định danh, cho phép chia sẻ vị trí thời gian thực giữa các thành viên và gửi thông báo đẩy (Push Notification) từ trưởng đoàn.

2.2 Hỗ trợ an toàn và tình huống khẩn cấp

- **Tự động hóa theo GPS:** Phát triển tính năng tự động đánh dấu hoàn thành địa điểm dựa trên tọa độ GPS và tái cấu trúc lộ trình (Re-routing) ngay lập tức nếu người dùng đi chệch hướng hoặc bỏ lỡ một điểm đến.
- **Bảo vệ người dùng:** Xây dựng lớp bảo vệ người dùng thông qua việc cung cấp danh bạ khẩn cấp địa phương, tính năng gửi tín hiệu SOS kèm vị trí và chỉ dẫn đến cơ sở y tế gần nhất khi phát hiện sự cố.

2.3 Hoàn thiện hậu hành trình

Phát triển các công cụ tổng kết chi phí thực tế so với dự kiến và tự động tạo "Bộ sưu tập kỷ niệm" để người dùng dễ dàng chia sẻ hành trình lên các nền tảng mạng xã hội.

3 Tài liệu tham khảo

Tài liệu tham khảo

- [1] Flutter. [Flutter Documentation](#), 2023.
- [2] Material.io. [Material Design 3 Guidelines](#), 2021.
- [3] Dart Language. [Dart Language Guide](#), 2023.
- [4] UX Design. [Glassmorphism Design Patterns](#), 2021.
- [5] Node.js Foundation. [Node.js](#), 2024.
- [6] Express.js. [Express Web Framework](#), 2023.
- [7] MongoDB. [MongoDB Documentation](#), 2024.
- [8] Software Architecture. [Microservices Architecture Pattern](#), 2024.
- [9] REST API. [RESTful API Design Best Practices](#), 2023.
- [10] Mongoose. [Mongoose ODM](#), 2024.
- [11] dcodeIO. [bcryptjs - Password Hashing](#), 2023.
- [12] Flutter Community. [Flutter packages](#), 2023.
- [13] Sebastián Ramírez. [FastAPI Documentation](#), 2023.
- [14] OpenAI. [OpenAI API Documentation](#), 2024.
- [15] Pydantic authors. [Pydantic Documentation](#), 2024.
- [16] IETF. [JSON Web Token \(RFC 7519\)](#), 2015.
- [17] Docker. [Docker Compose Documentation](#), 2024.